

Project Report: E-Commerce Platform on AWS Date: May 15, 2026

1. High-Level Overview This report summarizes the technical implementation of an automated deployment for a containerized e-commerce application. The system is built on modern DevOps principles, leveraging Infrastructure as Code (IaC), Configuration Management, and a fully automated CI/CD pipeline to deploy to Amazon Web Services (AWS).

The architecture is designed for security, scalability, and high availability. Application servers are placed in private subnets, accessed only through a load balancer and a secure bastion host, demonstrating a strong security posture.

2. Core Technologies The project utilizes the following key technologies:

Cloud Provider: Amazon Web Services (AWS) Infrastructure as Code: Terraform (~> 5.0) Configuration Management: Ansible (ansible-core==2.16.14) Containerization: Docker & Docker Compose (v2.20.0) CI/CD: GitHub Actions Application Stack: Node.js (in container), MongoDB (in container), Nginx (in container) 3. AWS Infrastructure Architecture (terraform/) The Terraform configuration defines and provisions a secure and resilient cloud environment.

Networking:

A Virtual Private Cloud (VPC) (10.0.0.0/16) provides network isolation. Two public subnets and two private subnets are created across different Availability Zones to ensure high availability. An Internet Gateway provides outbound and inbound internet access for public resources. A NAT Gateway in a public subnet allows instances in private subnets to access the internet for software updates without being directly exposed.

Security:

A Bastion Host (t3.micro) is deployed in a public subnet to act as a secure jump host for SSH access. Its security group (ecommerce-bastion-sg) allows SSH from 0.0.0.0/0 for CI/CD access. Application EC2 instances are placed in private subnets, preventing any direct internet access. The Application Load Balancer (ALB) security group (ecommerce-alb-sg) allows public HTTP/HTTPS traffic. The Web Server security group (ecommerce-web-sg) is highly restrictive, only allowing HTTP traffic from the ALB and SSH traffic from the Bastion Host. Compute & Load Balancing:

An Application Load Balancer distributes incoming traffic across the web server fleet. A configurable number of EC2 instances (var.instance_count) run the application. They are attached as targets to the ALB's target group. 4. Server Configuration & Deployment (ansible/deploy.yml) The Ansible playbook automates the complete setup of the application servers.

System Preparation:

The playbook begins by elevating privileges (become: true). It updates all system packages on the Amazon Linux 2 instances using yum. Docker & Docker Compose Installation:

It installs the Docker engine using amazon-linux-extras. It downloads and installs a specific version of Docker Compose (v2.20.0) to ensure build consistency. The docker service is started and enabled to run on boot. The ec2-user is added to the docker group to run Docker commands without sudo. A connection reset (meta: reset_connection) is correctly used to apply this group change immediately. Application Deployment:

The application source code (app/), docker-compose.yml, and nginx.conf are copied to the server. File ownership is set to ec2-user. The application stack is launched using the docker-compose up -d --build command. This builds the application image and starts the nginx, app, and mongodb services in detached mode. Finally, docker image prune -f is run to clean up dangling images and save disk space. 5. CI/CD Automation Pipeline (.github/workflows/pipeline.yml) The GitHub Actions workflow is the core of the project's automation, orchestrating the entire deployment from code push to running application.

Trigger: The pipeline runs on any push to the main branch and can also be triggered manually (workflow_dispatch).

terraform Job:

Provisions all AWS infrastructure defined in the terraform/ directory. On success, it exports the Bastion Host's public IP and the web servers' private IPs as job outputs. ansible Job:

This job depends on the successful completion of the terraform job. It dynamically generates an Ansible inventory file. Crucially, it configures Ansible to use the Bastion Host as a ProxyCommand (jump host), enabling secure connections to the private web server instances. It installs Python 3.8 on the target instances for Ansible compatibility. It executes the ansible/deploy.yml playbook to configure the servers and deploy the application. destroy Job:

A safety measure is included: this job only runs when manually triggered. It executes terraform destroy, which cleanly removes all AWS resources created by the pipeline, preventing orphaned resources and unnecessary costs.